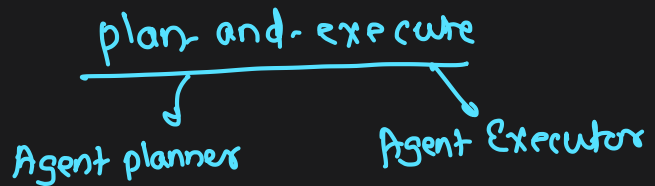
Multi-Agent Collaboration

Agent orchestration

microservices architecture

Introduction

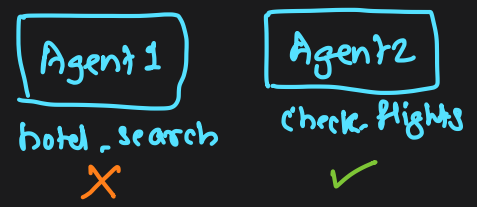
- Multi-Agent Collaboration is a design pattern in Agentic AI where **multiple autonomous, specialized agents work together** — through communication, coordination, and negotiation — to solve a problem that a single agent cannot handle effectively alone
- Each agent typically has:
  - A **role** (e.g., researcher, coder, critic, planner)
  - Its own **tools** and **memory**
  - A **goal** or sub-goal
  - The ability to **communicate** with other agents



- The core idea here is to **divide a complex task into sub-tasks, assign them to specialized agents, and orchestrate their interaction** to produce a coherent final result

Why Use Multi-Agent Collaboration?

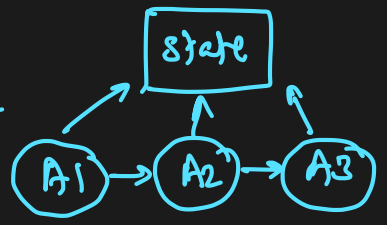
- **Specialization:** Each agent focuses on one domain (e.g., retrieval, coding, validation), improving quality
- **Modularity:** Easier to build, test, and replace individual agents
- **Scalability:** Add more agents to handle more workload or new capabilities
- **Robustness:** If one agent fails, others can compensate or retry
- **Parallelism:** Independent sub-tasks run concurrently, reducing latency
- **Emergent Intelligence:** Collective reasoning often outperforms a single LLM
- **Separation of Concerns:** Planning, execution, and critique are decoupled



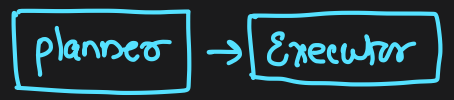
Key Components

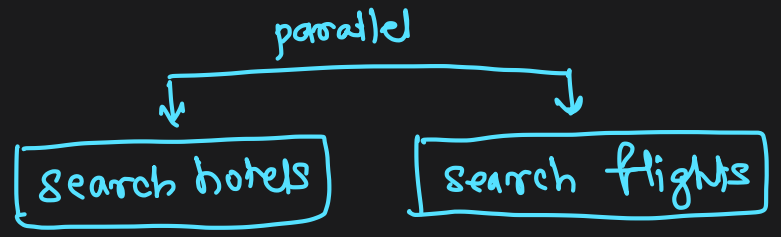
- **Agents** — Autonomous units with role, goal, tools, memory, and policy
- **Communication Channel** — How agents exchange messages (shared memory, message bus, chat)
- **Orchestrator / Coordinator** — Decides who acts when (static or dynamic)
- **Task Decomposer** — Splits the main goal into sub-tasks
- **Shared State / Blackboard** — Common workspace for intermediate results
- **Environment / Tools** — External APIs, databases, code executors
- **Evaluation / Critic** — Validates outputs and triggers refinement
- **Termination Condition** — When the task is considered complete

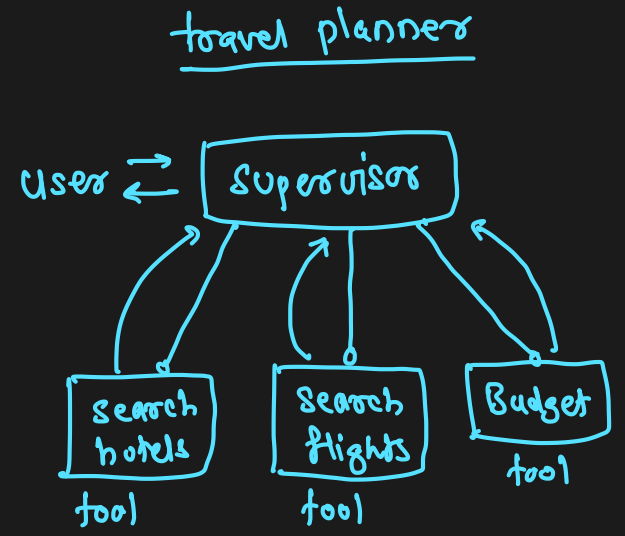
Major Orchestration Architectures



- Sequential Orchestration
- Concurrent / Broadcast (Parallel) Orchestration
- Hierarchical Orchestration
- Handoff / Routing Pattern
- Group Chat / Debate Pattern
- Blackboard Pattern
- Market / Auction-Based
- Competitive / Adversarial
- Supervisor Pattern







Communication Mechanisms



- **Message Passing:** Direct agent-to-agent messages (like actors)
- **Shared Memory / Blackboard:** Common store all agents read/write
- **Publish–Subscribe:** Agents subscribe to topics of interest
- **Structured Protocols:** JSON schemas, FIPA-ACL-like contracts
- **Natural Language Chat:** Free-form LLM dialogue (AutoGen, CrewAI)
- **Message content types:** requests, informs, proposals, critiques, confirmations, refusals

Coordination and Control Strategies

- **Centralized:** One orchestrator decides everything
- **Decentralized:** Agents self-organize via rules or negotiation
- **Hybrid:** Central planner + autonomous workers
- **Contract Net Protocol:** Manager announces task → agents bid → best bidder wins
- **Voting / Consensus:** Majority or weighted vote for decisions
- **Role-based:** Fixed roles (Planner, Executor, Critic, Verifier)

Design Considerations and Trade-offs

- **Cost:** More agents = more LLM calls = higher cost
- **Latency:** Parallelism helps; coordination adds overhead
- **Complexity:** More agents = harder debugging
- **Reliability:** Redundancy helps; cascading failures hurt
- **Autonomy vs. Control:** Free agents risk drift; tight control limits flexibility
- **Communication Overhead:** Too many messages slow the system
- **Convergence:** Debate/chat patterns may never terminate
- **State Consistency:** Shared memory needs synchronization

Frameworks and Implementations

- **AutoGen :** Conversational multi-agent, group chat → MS agents framework
- **CrewAI:** Role-based crews with sequential/hierarchical processes
- **LangGraph:** Graph-based orchestration with state
- **MetaGPT:** SOP-driven software team agents
- **CAMEL:** Role-playing agent dialogue
- **Swarm (OpenAI):** Lightweight handoff-based agents
- **AgentVerse:** Simulation of collaborative agents
- **ChatDev:** Multi-agent software development

- Google ADK →
- OpenAI Agent framework

Real-World Use Cases

- **Software development:** Planner + coder + tester + reviewer
- **Research:** Retriever + summarizer + critic
- **Customer support:** Triage → specialist → escalation
- **Content creation:** Researcher → writer → editor → fact-checker
- **Finance:** Analyst + risk + compliance agents
- **Robotics:** Swarm coordination
- **Game AI:** Adversarial and cooperative teams
- **Healthcare:** Diagnostic + specialist + safety agents